Recovering the Ternary Penalty in Masked Diffusion Language Models: What Adds Capacity, What Only Reorganizes, and What Repetition Costs

Daniele Scaratti
Independent Researcher
scaratti.daniele.research@proton.me

Abstract

Low-bit quantization of diffusion language models has meant quantizing after training. We train masked diffusion models with ternary weights from the first step instead, and ask what that penalty is and what buys it back. Post-training quantization is not the alternative at this precision: it collapses, by a factor of 26 for round-to-nearest and 4.7 even for calibrated GPTQ, while native training survives at a penalty of roughly 20% perplexity against a full-precision twin trained on the same tokens, in the same order, under the same masks. The paper's central result is a predictive rule for recovery. Across a controlled eight-arm ablation, only interventions that *add* something to the run recover the gap, information via distillation from the twin or parameters via a learnable per-channel scale (70% of the gap at 27M, at 0.07% of memory), while every intervention that merely *reorganizes* the existing training (thresholds, schedules, self-conditioning, variance reduction) leaves it untouched. The rule is falsifiable and we test it: it predicts, and the data confirm, that optimization-level tricks cannot move a penalty that is a fitting deficit under the constrained grid rather than an optimization failure. Two supporting measurements sharpen the picture. The bare penalty is empirically a near-constant offset, about 0.19 nats per masked token in our 27M-to-341M, undertrained regime, whether tokens are fresh or a fixed pool repeated twenty times; and it concentrates where prediction depends on rich context. Recovery, however, is scale- and mode-dependent: at 341M it works applied post-hoc to a finished model and not from the first step. We report three pre-registered predictions that went zero for three, release code, models, and every raw number, and close with a deployment accounting whose conclusion is that the win is memory, not latency, and that without a memory constraint one should train FP16.

1 Introduction

What does ternary precision cost a masked diffusion language model, and how much of that cost can a practitioner with a finite corpus buy back? This paper answers the question by measurement. Every ternary model we train is paired with a full-precision twin that sees the same tokens in the same order under the same masks, so the gap between the two is the price of precision and nothing else; the price is then followed across token budgets (150M to 4B), model sizes (27M to 341M parameters), and, because a mid-resource pipeline exhausts its corpus and must repeat it, across up to twenty epochs of a fixed pool.

The question has two halves, and the paper is organized around them. The first half is whether native ternary training is viable at all, and at what cost. Ternary quantization-aware training (QAT) works for autoregressive LLMs (Ma et al., 2024; 2025; Chen et al., 2024) and for image diffusion transformers (Lu et al., 2024), but the masked-denoising objective differs from both in ways that could plausibly break it: it is a reweighted ELBO whose $1/t$ term already carries high variance (Section 2.1), and stacking on it the jagged optimization of ternary QAT, where weights flip sign as latent values cross rounding boundaries, is not obviously survivable. Every low-bit diffusion-LM result we can find quantizes *after* training (Zhang et al.,

2025; Xu & Yang, 2025; Lin et al., 2025), and at 3 to 4 bits that is reportedly a good option (Conzelmann et al., 2026). At 1.58 bits it is not: Section 5 shows post-training quantization collapses by a factor of 26, calibration recovers only a fraction of the loss, and no post-training method we evaluate, round-to-nearest or calibrated GPTQ, yields a working model, and native training does (we do not evaluate incoherence-processing methods such as QuIP or additive-codebook methods, which are the strongest sub-2-bit PTQ family).

The second half is what the working model costs and what buys the cost back. The penalty is real and it moves: it grows with the token budget (Section 6), is recovered only by interventions that add information or parameters to the run, never by rearranging what the run already has (Section 7), and concentrates in predictions that depend on rich context (Section 8). On repeated data, the regime a finite corpus forces, the same penalty follows the same trajectory and flattens near 20%, and the same recovery recipe holds through the three-to-five epoch window where repetition pays most (Section 9).

Scope of the claim. Two adjacent results already exist. Ternary QAT for *language* models is established (Ma et al., 2024; 2025; Chen et al., 2024), and ternary QAT for *diffusion* models is established for image DiTs (Lu et al., 2024). The novel unit is their combination, a masked-diffusion objective with BitNet-style ternary weights quantized in the training loop, plus the recovery, mechanism, objective-comparison, and repetition results, which are new for any bit width.

Contributions.

- **The central result: a predictive taxonomy of recovery.** An eight-arm ablation shows the ternary penalty is recovered only by interventions that *add* information or parameters (distillation from the twin, a per-channel scale; 70% of the gap at 27M at 0.07% of memory), never by ones that *reorganize* the existing training, and that stacking reorganizations hurts (Section 7). The split is registered a priori and used predictively, correctly separating what can move a fitting deficit from what cannot.

- A mechanistic account of where the penalty lives: a matched-objective control places it in the weights rather than the objective, and a mask-rate decomposition, in absolute nats, shows it concentrated where prediction depends on rich context (Section 8).

- That recovery is scale- and mode-dependent: at 341M the recipe recovers 41 to 49% applied post-hoc to a finished model, on validation, held-out test, and out-of-domain text, and nothing measurable applied from the first step (Section 7.1).

- A measurement of what native training buys over post-training quantization: round-to-nearest multiplies perplexity by 26 and calibrated GPTQ still by 4.7, while native QAT costs a few percent, on identical weights (Section 5). This, and the first well-behaved native ternary masked diffusion LM (Section 12), are what make the recovery question worth asking.

- Two supporting measurements: the bare penalty is empirically a near-constant offset (about 0.19 nats per masked token in our undertrained regime, fresh tokens or a pool repeated twenty times; an empirical regularity, not a fitted law, Section 10), and masked diffusion extracts value from repetition through twenty epochs at both precisions (Section 9).

- Three pre-registered technique predictions that went zero for three, reported with the misclassification analysis, plus a replicated free win (per-epoch learning-rate re-warming) that emerged from the failure (Section 9.4); and a task-level account showing ternary costs 7 to 9% of infilling accuracy while free generation fails identically at both precisions (Section 11).

- A deployment accounting that includes its own bad news: the win is memory (5.4×), not latency, and a practitioner without a memory constraint should train FP16 (Section 13).

2 Background

2.1 Masked diffusion language models

We use the absorbing-state (masked) discrete diffusion formulation of MDLM (Sahoo et al., 2024) and MD4 (Shi et al., 2024), which descends from D3PM (Austin et al., 2021) and underpins the LLaDA (Nie et al., 2025) and Dream (Ye et al., 2025) families. For a clean sequence x_0 of length L, draw a corruption level $t \sim \mathcal{U}(\varepsilon, 1]$ and replace each token independently with a reserved `[MASK]` symbol with probability t:

$$x_t^{(i)} = \texttt{[MASK]} \text{ w.p. } t, \quad x_0^{(i)} \text{ w.p. } 1 - t. \tag{1}$$

A bidirectional denoiser p_θ predicts the originals at masked positions. The objective is a Monte-Carlo negative ELBO,

$$\mathcal{L}(\theta) = \mathbb{E}_{t, x_0, x_t}\Big[\tfrac{1}{t}\tfrac{1}{L} \sum_{i : x_t^{(i)} = \texttt{[MASK]}} -\log p_\theta(x_0^{(i)} \mid x_t)\Big]. \tag{2}$$

This is BERT-style masking with two differences that matter. The masking ratio is random per example rather than fixed, so the model denoises at every corruption level; and the $1/t$ reweighting is what makes the estimator a proper continuous-time ELBO rather than a heuristic. It is not a tunable knob, and its cost is variance, since the weight diverges as $t \to 0$, which is why the draw is clamped at $t \geq \varepsilon$. Generation is iterative unmasking, not left-to-right decoding, so the denoiser is bidirectional and there is no KV cache to amortize. This last fact returns in Section 13: it makes per-step compute, and therefore weight-matmul throughput, matter more for diffusion LMs than for autoregressive models of the same size.

2.2 Ternary quantization-aware training

Following BitNet b1.58 (Ma et al., 2024), each linear layer stores weights in $\{-1, 0, +1\}$ scaled by a single per-tensor factor $s = \text{mean}(|W|)$:

$$\widetilde{W} = \text{clamp}\big(\text{round}(W/s), -1, 1\big) \cdot s. \tag{3}$$

Activations are quantized to per-token int8 by an absmax scale. Both quantizers are non-differentiable, so gradients reach the latent full-precision weights through a straight-through estimator (Bengio et al., 2013): the forward uses $\widetilde{W}$, the backward pretends the rounding was the identity. The scheme descends from BinaryConnect (Courbariaux et al., 2015) and XNOR-Net (Rastegari et al., 2016), and the three-level grid from Ternary Weight Networks (Liu et al., 2023); later refinements learn the quantizer (Esser et al., 2020; Choi et al., 2018) or distil from a full-precision teacher (Du et al., 2024).

The latent weights are not the model. Under STE, W is not an approximation of $\widetilde{W}$; it is a gradient accumulator whose only job is to determine, through Eq. 3, which of three values each weight takes. Nothing penalizes $\|W - \widetilde{W}\|$. Training is free to move W arbitrarily far from the grid, provided the projection onto it computes a good function. Post-training quantization has no such freedom: it receives a fixed W and must find the best ternary approximation of *those* weights. At 4 to 8 bits the distinction is mild; at 1.58 bits it is not, and Section 5 measures how far the two part company.

Why normalization is load-bearing. A normalization (SubLN) precedes every quantized matmul, inside the quantized layer. Per-token absmax activation quantization is stable only if activation magnitudes are controlled, otherwise a single outlier token sets the scale for the whole row. We also apply QK-norm, which is not part of the b1.58 recipe, because ternary weights destabilize attention logits (Section 12).

3 Method

The denoiser is a bidirectional transformer with RoPE and a SwiGLU feed-forward block. Every linear inside the blocks (four attention projections, three FFN projections) is a ternary `BitLinear` following Eq. 3, with SubLN placed inside the layer. At 341M total parameters, 308M (90%) are ternary.

What stays full precision, and why. The token embedding, the LM head, and all normalization scales remain full precision: three levels cannot separate 32k token identities, and ternarizing the embedding collapses quality. This is also where the memory accounting bites (Section 13): at this size the FP16 embedding table is larger than all 308M ternary weights combined.

The recovery levers. Beyond the baseline recipe we evaluate two capacity-adding components (Section 7). *Distillation*: a frozen FP16 twin, which we train regardless for the comparison, supplies a KL target over the masked positions of the same batch, at the cost of one extra teacher forward per step (about half again the step time in our runs) and no extra unique tokens. *Per-channel scale*: a learnable multiplier per output row, applied after the matmul and outside the quantizer, so it cannot alter the zero-bin, at a cost of one FP16 vector per matrix (+0.07% memory). With every lever off, the model is bit-identical to the BitNet baseline, which is what makes each ablation attributable.

The comparison protocol. Every ternary model is paired with an FP16 twin of identical shape, seeing the same token stream in the same order, and, at validation, the same held-out windows under the same mask seed. The masked cross-entropy difference is therefore attributable to precision alone. Much of the confusion in the low-bit literature comes from baselines that differ in data, budget, or masks, producing a number that cannot be interpreted.

4 Experimental setup

Data. Italian subset (`ita_Latn`) of FineWeb-2 (Penedo et al., 2025), tokenized with a SentencePiece (Kudo & Richardson, 2018) unigram model (32k vocabulary, one reserved `[MASK]` id).

Why a 27M proxy. The eight-arm ablation runs at roughly 27M non-embedding parameters (34M total with the tied embedding) (d_{model}=448, 8 layers, 7 heads, d_{ff}=1216, sequence length 512), a scale chosen for three reasons. First, each arm finishes in roughly 40 minutes on a commodity GPU, so a controlled ablation with paired seeds is affordable, where the same sweep at 341M would cost tens of GPU-hours per arm. Second, the ternary–FP16 gap at this scale is already well above the noise floor (8.6% at 150M tokens, 21% at 300M), so the ablation resolves real effects rather than seed noise; below a certain size the gap disappears into the floor and the comparison becomes uninformative. Third, the ablation asks *which mechanism* recovers the gap, a qualitative question that a small ladder answers reliably, and we do not extrapolate its magnitudes: the 341M runs of Section 7.1 confirm the winning recipe at the target scale rather than assuming it. The 27M setting is a proxy, used as one, and never as a point on a scaling curve.

Noise model. Two runs of one configuration with everything except the seed held fixed land $\sigma_{\text{pair}} = 0.036$ masked-CE apart; the 3-seed replication of the winning arms gives a seed standard deviation $\hat{\sigma} = 0.013$, so σ_{pair} is a conservative single-pair estimate. One standard applies throughout: a single-run effect is noise below σ_{pair}; a difference of two independent single runs is noise below $\sqrt{2}\,\sigma_{\text{pair}} \approx 0.05$; a mean of n paired differences is judged against $\hat{\sigma}_{\text{diff}}/\sqrt{n}$ computed from the pairs themselves. The central PTQ result (3.26 masked-CE) is about 90 times σ_{pair} and is the only comparison in this paper for which the noise model is irrelevant.

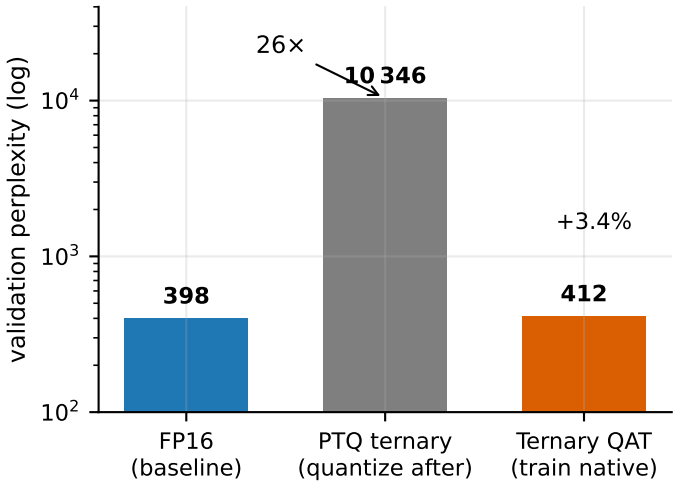

Figure 1. At 1.58 bits, *when* you quantize decides whether you have a model. All three share an architecture, a corpus, a token budget, and a validation set; the middle bar is the left bar's weights, rounded to $\{-1, 0, +1\}$ after training. Proxy setting (27M, 150M tokens).

5 Native training versus post-training quantization

At 1.58 bits, quantizing after training is not a substitute for training in ternary (Table 1; Figure 1), on identical weights, validation data, and mask seed.

	masked-CE	ppl	vs. FP16
FP16 baseline	5.9878	398.5	n/a
+ PTQ (RTN)	9.2444	10346.5	+2496%
Ternary (QAT)	6.0207	411.9	+3.4%

Table 1. Ternarizing a trained FP16 model after the fact multiplies perplexity by 26. Training the same architecture natively in ternary costs a few percent. Proxy setting (27M, 150M tokens). Rows 1 and 2 share weights exactly, so the 3.26 masked-CE penalty is attributable to quantization alone, roughly 90 times the noise floor, and independent of seed and learning rate. PTQ is round-to-nearest with the same per-tensor absmean scale used in training. Raw source: `results/raw/ptq_vs_qat_27m.csv`, run 2026-07-13, reproducible via `ptq_baseline.py`.

Why native training wins, measured in the weights. [1] QAT latent weights carry a *higher* weight-reconstruction error than FP16 weights (0.604 vs. 0.544 relative), yet perform far better once quantized, because QAT minimizes the loss of the quantized model, not $\|W - \widetilde{W}\|$. Training also restructures the residual: across all 56 quantized matrices the first singular direction of $W - \widetilde{W}$ carries 17.5% of the residual energy on QAT weights against 8.4% on FP16 weights, rising to 37.4% in the attention output projection (Figure 2). Ternary QAT pushes what it cannot represent into a low-dimensional subspace, which is also why the per-channel correction of Section 7 is more promising on QAT weights than on post-hoc quantized ones.

Calibrated PTQ does not escape the collapse. Round-to-nearest is the weakest PTQ baseline, so we also give post-training quantization its strongest practical form at this bit width: GPTQ (Frantar et al., 2023) with per-channel scales, activation ordering, and Hessian error compensation, calibrated on 128 masked sequences from the training distribution (Table 2). Every step of sophistication helps, and none of it is enough. Moving from per-tensor to per-channel scales cuts the RTN penalty from 17× to 10.6×; adding GPTQ's calibrated error compensation cuts it to 4.7×; native QAT costs 7.7%. Calibration recovers a factor

[1] Raw source: `results/raw/weight_analysis_27m.md`, `analyze.py` on the two 150M-token checkpoints.

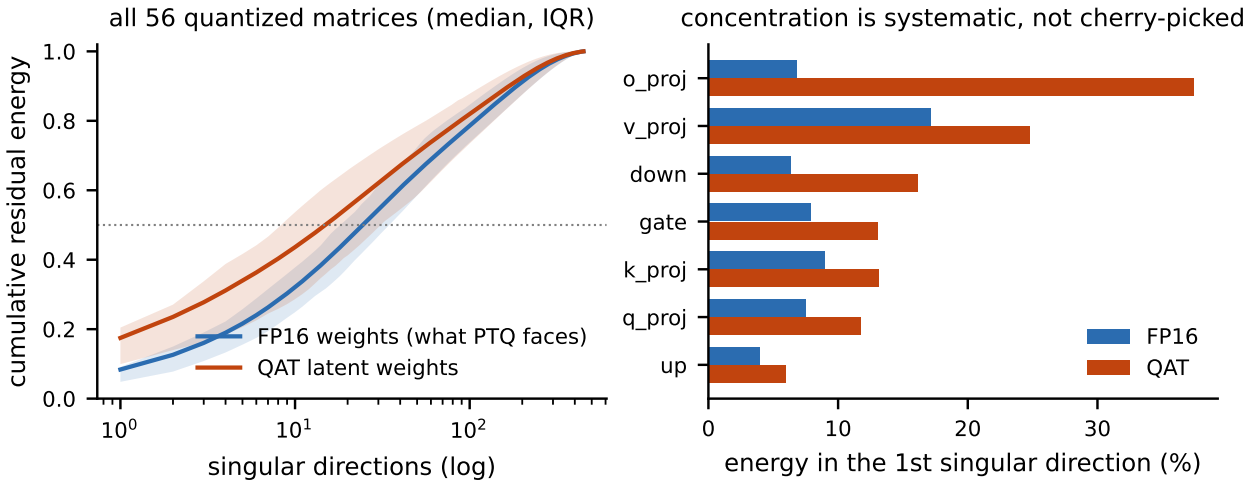

Figure 2. Why native training wins, measured in the weights. **Left:** cumulative energy of the quantization residual $W - \widetilde{W}$ across singular directions, over all 56 quantized matrices. On FP16 weights it is diffuse; on QAT latent weights the first direction alone carries 17.5% against 8.4%. **Right:** the effect holds for every matrix type, strongest in the attention output projection (6.8% → 37.4%). Training reorganizes the weights so that what cannot be represented is low-rank.

of a few and remains two orders of magnitude short of native training, consistent with reports that GPTQ-family methods degrade sharply below 2 bits (Chee et al., 2023). The conclusion of Table 1 survives its strongest challenger.

27M model	masked-CE	ppl	ppl vs. FP16
FP16 (as trained)	5.9714	392.0	1×
RTN per-tensor + A8	8.8129	6720.2	17.1×
RTN per-channel + A8	8.3311	4150.9	10.6×
GPTQ per-channel + A8	7.5158	1836.9	4.7×
QAT ternary (native)	6.0452	422.1	1.08×

Table 2. Calibrated ternary PTQ on the 27M proxy, one FP16 checkpoint, identical validation protocol across rows; all ternary rows also quantize activations to int8 (A8), so they differ only in how the weights reached the grid. GPTQ uses per-channel absmean scales, activation ordering, dampened Hessian error compensation, and 128 masked calibration sequences. Calibration buys a factor of a few; it does not prevent the collapse. (Absolute numbers differ slightly from Table 1, which used a different seed's checkpoint; the collapse is seed-independent.)

6 The penalty grows with tokens

The parameter-matched ternary penalty is 19.4% perplexity at 300M tokens (three paired seeds; 21.1% for the single default-LR run, 17.7% at best learning rate), up from 8.6% at 150M (blue curve in Figure 6, shown later alongside the repeated-data arms). This is the diffusion analogue of the known autoregressive result that quantization penalties grow with tokens-per-parameter (Kumar et al., 2025; Ouyang et al., 2025): the near-parity seen in undertrained runs is an artifact of being left of the scaling curve, not evidence ternary is free. A learning-rate control tempers but does not remove it: the fixed LRs were tuned at 150M tokens; at 300M the bracket we ran favors 3e-3 (5.9030), with both 6e-3 (5.9179) and the original 4e-3 (5.9316) worse, while FP16 does not improve off 2e-3, suggesting a best-LR gap of 17.7%. The individual LR deltas sit at or below the noise floor, so we report the honest range: the 300M-token gap is 17.7% to 21.1% depending on tuning (19.4% across three paired seeds at the default rate, the figure the recovery accounting uses). Under either reading the penalty grows across this budget range, and Section 10 will give the growth its shape: an offset still ramping toward a constant value it reaches near eight tokens per parameter.

7 What recovers the gap: addition, not redistribution

We group the interventions a priori by a single criterion, then test whether the grouping predicts which ones work; Table 3 reports the eight informative arms, and two further measured arms are documented in the project log and excluded here with cause (one ran at half batch size with a variance-reduction confound, one is a redundant subset of the stacked arm).

The criterion, stated so it can be applied without judgment calls: a training run is the tuple $(\theta, \mathcal{D}, \mathcal{S})$ of trainable parameters, training data with labels, and supervision signal. An intervention is *additive* if it enlarges the tuple, by adding parameters to θ or by adding to $\mathcal{S}$ information not derivable from $(\mathcal{D}, \text{labels})$ alone; it is *redistributive* if it changes only how the existing tuple is scheduled, weighted, or presented. **Additive** interventions do: distillation from the FP16 twin imports information (the teacher's soft targets carry strictly more supervision per token than the one-hot labels both runs share), and the learnable per-channel scale adds parameters (Section 3). **Redistributive** interventions do not: a TWN-style zero-threshold, the two-stage BitNet LR schedule with weight decay to zero, self-conditioning (Loopholing-style feedback of the previous hidden state), and stratified-t with antithetic-mask variance reduction all rearrange how the same data reaches the same parameters.

One objection deserves to be met head-on, because the criterion is doing real work there. At inference time a distilled ternary model has exactly the same degrees of freedom as the baseline ternary model, so if the taxonomy were about inference-time capacity, distillation would sit on the wrong side of it: it changes the training signal, not the network. That is precisely why we do not define the axis as capacity. The axis is what enters the run: the per-channel scale adds parameters, distillation adds information from outside the run's own (data, labels) pair, and the redistributive arms add neither. The prediction this makes is falsifiable and specific: if the ternary penalty were an optimization failure, the redistributive arms, which include exactly the schedule-, estimator-, and variance-level tools the QAT literature proposes for broken optimization, should recover part of it. If the penalty is instead a fitting deficit under a constrained grid, only the additive arms should move it.

The data lands entirely on the second reading (Figure 3, Table 3). Distillation (+50% of the gap, 3-seed) and the per-channel scale (+30%, single seed at $1.6\,\sigma_{\text{pair}}$, marginal alone but 3-seed and solid in combination) are the only arms clearing the noise floor. Every redistributive arm is within ±noise, and stacking all of them lands about 20% *below* the ternary baseline: raising the zero-threshold increases sparsity and removes expressive range precisely where the per-channel scale adds it, so the two compete for the same resource. Section 12 supplies the mechanistic reason: there is no instability for schedule- or estimator-level interventions to fix.

arm	Δ masked-CE	gap closed
distillation + per-channel	+0.124	70%
distillation	+0.089	50%
per-channel scale	+0.057	30%
two-stage LR	+0.029	15%
zero-threshold + TWN	+0.014	7%
self-conditioning	−0.011	0%
+ variance reduction	−0.016	0%
all arms except distillation	−0.035	−18%

Table 3. Improvement over the ternary baseline at 27M / 300M tokens. The two winning arms and the baseline are means over 3 paired seeds: ternary 5.9175, distillation 5.8285 (±0.010), distillation+per-channel 5.7940 (±0.013); the gap to FP16 (5.7399) is 0.178, and the recovered fractions (50%, 70%) exceed the seed standard deviation (0.013) by an order of magnitude, and the single-pair noise floor (0.036) by 2.5 to 3.4 times, so the recovery is statistically solid rather than a single-run artifact. The two additive arms are near-orthogonal: their single gains (+0.089, +0.057) sum to +0.146, measured jointly +0.124 (85%), consistent with acting on orthogonal axes (training target vs. static parameters). Other arms are single seed.

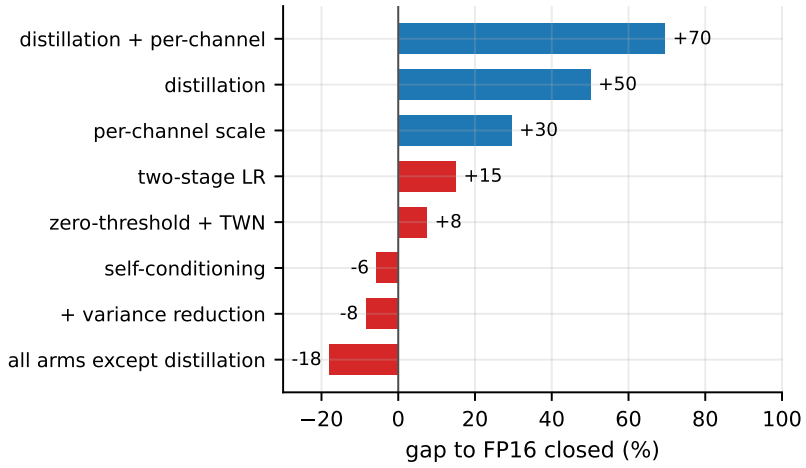

Figure 3. The eight ablation arms at 27M, sorted by the fraction of the ternary-to-FP16 gap each closes (positive = closer to FP16 than the ternary baseline). Blue bars are the additive arms (they bring information or parameters the baseline run lacks: distillation from the twin, the learnable per-channel scale, and their combination); red bars are the redistributive arms (same data, same parameters, rearranged: schedules, thresholds, self-conditioning, variance reduction). Only additive arms clear the noise floor, and stacking every arm except distillation lands below the baseline. The two winning arms and the baseline are 3-seed means; the rest are single seed.

The clean recipe, distillation from the free FP16 twin plus a per-channel scale, recovers 70% of the gap at +0.07% memory. The teacher is not circular: it is used only at training time, and at inference we ship only the ternary model. This also answers the QAT-versus-PTQ question of Section 5 in its strongest form: the best route is neither post-quantization (which collapses) nor bare QAT (which leaves 21% on the table), but QAT with the full-precision twin as a distillation teacher.

7.1 Recovery at 341M parameters

The bake-off above is a 27M proxy. We confirm the recipe at the project's target scale, 341M parameters, against the trained ternary baseline (ppl 132.4 at training time, 146.2 under the common re-evaluation of Table 4) and its FP16 twin (Table 4), in two regimes that answer two different questions. *Continued distillation* takes the already-trained ternary model, adds a per-channel scale (initialised at identity), and continues for a further 1B tokens at a small learning rate with the FP16 twin as teacher; it answers the practitioner's question, "I have a trained ternary model with a known degradation, how much of it can I recover without retraining?". *From-scratch distillation* trains a fresh ternary model with the recipe from step 0 at the baseline's full token budget, which is the parity comparison for the scaling claim.

341M model	masked-CE	ppl	vs. FP16
FP16 twin (ceiling)	4.8100	122.7	ceiling
Ternary baseline	4.9852	146.2	+19.2%
+ continued distil.	4.9125	136.0	+10.8%
+ from-scratch recipe	4.9878	146.6	+19.5%

Table 4. Recovery at 341M parameters, all rows re-evaluated under one common protocol (identical seeded validation windows and masks, sequence length 1024), which is why the absolute numbers differ slightly from the training-time figures quoted elsewhere; the relative gap matches (19.2% here, 19.4% at training time). Continued distillation recovers a trained ternary model post-hoc; the from-scratch recipe is the parity comparison. Both distil from the FP16 twin we train anyway.

No selection bias: held-out test and out-of-domain replication. The validation split selects every best checkpoint, so we re-score the 341M models on a true test split (an adjacent held-out slice never seen by any run and never used for selection) and on an out-of-domain sample (5M tokens of fresh Italian Wikipedia,

same tokenizer). Both conclusions replicate: the ternary gap reads 19.2% on validation, 20.2% on test, and 21.1% out of domain; continued distillation recovers 41%, 48%, and 49% of it respectively. The recovery is, if anything, slightly larger away from the selection split, and the out-of-domain number shows the distilled model did not overfit the teacher's domain.

The 27M finding holds directionally at scale. Continued distillation, 1B extra tokens (a 25% budget extension) at a small learning rate with a per-channel scale costing 0.49 MB, recovers 41% of the ternary-to-FP16 gap (masked-CE 4.9852 to 4.9125 against a 4.8100 ceiling) and cuts the perplexity penalty from 19.2% to 10.8%. This is the post-hoc answer: nearly half the degradation of an already-trained ternary model is recoverable without retraining it. The from-scratch row tests the 27M recipe at parity, and it fails to transfer: trained from step 0 with distillation and the per-channel scale for the full 4B tokens, the 341M student lands within noise of the bare ternary baseline (masked-CE 4.9878 against 4.9852, a +0.003 difference under a 0.05 propagated threshold), against the 70% recovery the identical recipe delivers at 27M at matched tokens-per-parameter, and 47% at an intermediate 110M-parameter rung (a lower bound: the rung's recipe arm ran at an untuned learning rate, and the fraction is computed against the LR-controlled baseline). The breakdown is therefore located between 110M and 341M rather than being a smooth degradation: past that scale the teacher's signal helps a mature ternary model and does not help a ternary model build itself, and recovery is post-hoc or not at all. The distillation literature documents a related but distinct failure, students unable to absorb teachers too far ahead in *parameters* (Cho & Hariharan, 2019; Mirzadeh et al., 2020), a gap that closes as the student grows; ours is the opposite in that coordinate (teacher and student are parameter-matched, and the failure *appears* as the student grows), so we treat it as a representational gap, three-valued weights mid-construction against a full-precision signal, not the parametric capacity gap of that literature. Stated without cushioning: the claim that the from-scratch recipe scales does not survive this measurement; what scales is post-hoc distillation. The failure is a single-configuration result, the 27M recipe transferred verbatim (LR 4e-3, KD weight 0.5, temperature 2), because a hyperparameter sweep at 341M costs roughly $80 per point against the $4 the entire 27M ablation cost; alternative schedules, a short FP16 warmup before quantization, a higher distillation weight, a lower learning rate, may help, but the 27M recipe was transferred verbatim to test transferability rather than tuned to force recovery, and we report the clean transfer failure rather than guess. Two mechanisms are compatible with the data and the identical hyperparameters working at both smaller scales; we state them as registered working hypotheses, not conclusions, since nothing in our measurements discriminates between them yet. One is depth: at 24 layers the distillation gradient crosses twice as many quantized layers as at the proxy scales, each adding straight-through estimator noise, so the imitation signal arrives degraded exactly while representations form. The other is maturity: the mask-rate decomposition of Section 8 shows the penalty concentrated in fine-grained context exploitation, a capability models build late, so the teacher presses hardest on what the mid-construction student does not yet have. The cheap discriminating experiment is a short full-precision warmup before quantization begins: if it rescues from-scratch distillation the maturity account wins, if only re-tuned optimization does then the failure was hyperparameters after all. We leave it registered as the first experiment of any follow-up, with its design committed to the repository. (The rung's initially anomalous bare gap was an artifact of an untuned learning rate; an LR-control arm resolves it to 15.5% at 4 tokens per parameter, inside the ramp region Eq. 4 predicts.)

The practical recipe therefore simplifies: train the ternary model bare, then distill from the twin, which is also the cheaper schedule (the distillation phase pays the teacher forward for a quarter of the tokens rather than all of them).

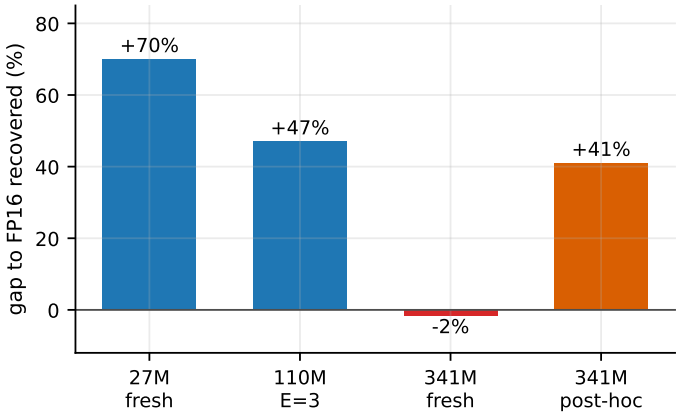

Figure 4. The fraction of the ternary-to-FP16 gap recovered by the distillation-plus-scale recipe, across scales and application modes. Applied from the first training step (blue and red bars) the recipe recovers two thirds of the gap at 27M, roughly half at 110M (lower bound, see text), and nothing at 341M; applied to the finished 341M model (orange bar) it recovers 41%. The failure is not a smooth degradation with scale but a breakdown between 110M and 341M.

8 The penalty is in the weights, not the objective

The recipe says what recovers the penalty; the next two controls ask where it lives. Holding architecture and optimizer fixed and swapping only the objective (causal mask plus next-token loss), the ternary penalty is 23.6% autoregressive versus 21.1% masked-diffusion (single runs; the two offsets, 0.212 and 0.191 nats, differ by less than the propagated noise floor $\sqrt{2}\,\sigma_{\text{pair}} \approx 0.05$) (Figure 5), effectively the same. The penalty is intrinsic to the ternary weights; the objective is innocent, and the recovery recipe is needed under both.

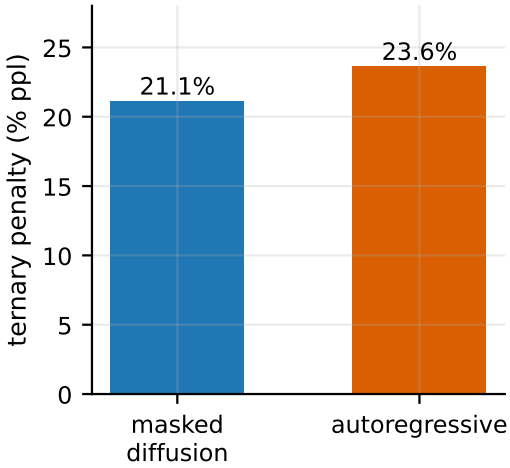

Figure 5. Matched architecture and optimizer: the ternary penalty is the same under masked diffusion and autoregression, locating it in the weights rather than the objective.

A trap we record. An earlier mixed-optimizer comparison suggested a 2.7× advantage for autoregression (a 21.1% diffusion penalty against an apparent 7.8% autoregressive one). It was an artifact of comparing a Muon-trained FP16 ceiling against an AdamW-trained ternary floor: the stronger optimizer lowered the FP16 baseline and inflated the apparent objective effect. Under a matched-optimizer control it vanishes. We report it because comparing arms trained with different optimizers is an easy and invisible way to manufacture a result, and it is adjacent to the same discipline that the paired-twin protocol enforces elsewhere.

Where in the denoising process the penalty binds. Decomposing the 341M validation loss by mask rate locates the penalty mechanistically: at heavy masking ($t \in [0.9, 1.0)$, near-unconditional prediction) the ternary model is within 5% of its twin, while at light masking ($t \in [0.1, 0.4)$, reconstruction from rich context) the gap is 28 to 31%. The effect is not a denominator artifact of easier bins having lower absolute loss: the absolute cross-entropy gap itself falls from 0.25 nats at light masking to 0.05 at heavy masking, so the ternary model pays most where prediction draws on rich context. Ternary weights reproduce the corpus statistics that dominate context-poor prediction, and lose the fine-grained context exploitation that rich conditioning rewards. Continued distillation recovers a nearly constant 37 to 44% of the gap in every bin: the teacher's signal repairs the model uniformly across the process rather than targeting a regime. (Raw source: `results/raw/decompose_t.json`.)

9 The penalty under repeated data

The account so far prices ternary training on fresh tokens. A mid-resource pipeline does not have that luxury: our Italian corpus yields 4B usable tokens, about 12 per parameter at 341M, and the only lever left after the corpus is exhausted is to repeat it. For masked diffusion the lever is unusually strong, because every pass re-masks the text and presents a new prediction problem (Ni et al., 2025; Prabhudesai et al., 2025), where for autoregressive models repeated data holds its value for roughly four epochs and decays toward worthlessness by sixteen (Muennighoff et al., 2023). For a ternary model the lever carries a specific risk: if repeated tokens act like fresh tokens, every extra epoch widens the gap of Section 6. This section adds the repeated-data column to the price list: the same paired-twin protocol, on a fixed unique pool, from one to twenty epochs.

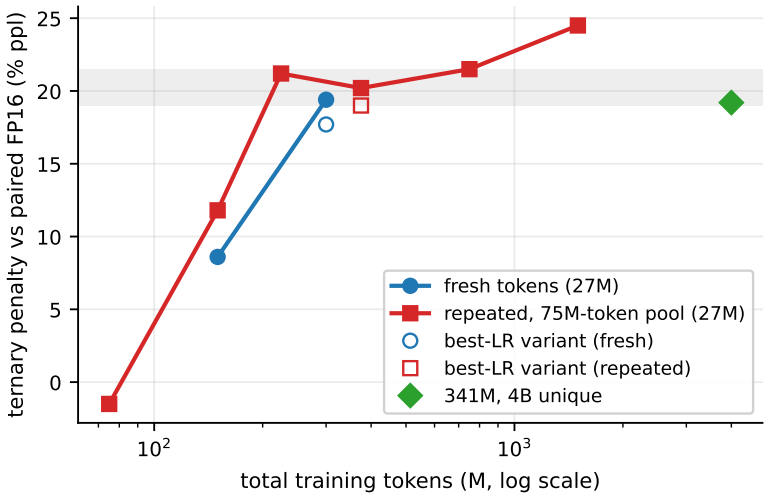

Figure 6. The master measurement, every point a paired ternary/FP16 twin under one validation protocol. The penalty follows the same trajectory whether tokens are fresh (blue) or repeated views of a 75M-token pool (red), and flattens near 20%: the same value the 341M model reaches with 54× the unique data (green). Open markers are best-LR variants.

9.1 The ladder: one pool, one to twenty epochs

Everything inherits the protocol of the earlier sections: 27M-parameter masked diffusion transformers (BitNet-style ternary QAT or FP16), Italian FineWeb-2, a 32k SentencePiece vocabulary, identical seeded validation windows and masks for every arm, and the measured run-to-run noise floor of 0.036 masked-CE. The proxy scale is deliberate and tested: in the earlier sections every mechanism found at 27M reproduced directionally at 341M.

The ladder fixes a single **unique pool**, the first 75M tokens of the training split, and varies only the budget: $E \in \{1, 2, 3, 5\}$ epochs, so total tokens range from 75M to 375M. Masks are drawn fresh at every step (the training default), so each epoch is a new view of the same text. Three tracks per epoch count: the FP16 twin (which doubles as the epoch-matched distillation teacher), bare ternary, and ternary with the recovery recipe (distillation from the epoch-matched twin, KD weight 0.5, temperature 2, plus a learnable per-channel scale). A learning-rate control at E=5 re-checks the optimum at the largest budget, since Section 6 found the optimum drifts with budget. Fourteen base arms in roughly thirteen hours on a single RTX 3090; the deep-epoch and technique arms add a further GPU-day.

9.2 The penalty tracks the total budget, then flattens

Three observations from Table 5 and Figure 6.

Repetition works at both precisions. The FP16 twin improves on every pass: 6.37 → 6.04 → 5.87 → 5.75, and five epochs of the 75M pool (375M total) land within noise of the FP16 baseline of Section 7 trained on 300M *fresh* tokens (5.7399). For masked diffusion, a repeated token is worth almost a fresh one, confirming the super-data-learner reports (Ni et al., 2025; Prabhudesai et al., 2025) at 27M scale. The ternary model improves on every pass as well (6.36 → 5.94), and its five-epoch value lands 0.019 CE from the fresh-300M ternary baseline of Section 7 (5.9358 vs. 5.9175, inside the noise floor); repetition is not what breaks at low precision.

val masked-CE	E=1	E=2	E=3	E=5
FP16 twin	6.3716	6.0442	5.8730	5.7517
ternary	6.3561	6.1553	6.0654	5.9358
ternary + recipe	6.3036	6.0640	5.9518	5.8315
gap (% ppl, from masked-CE)	−1.5	+11.8	+21.2	+20.2
recipe recovers	>100%	82%	59%	57%
residual (% ppl)	−6.6	+2.0	+8.2	+8.3

Table 5. The ladder at 27M: one 75M-token unique pool, E epochs, identical seeded validation. At E=1 (2.8 tokens per parameter) ternary matches FP16 within noise and the recipe lands 0.068 CE below the twin, an effect of the teacher's soft targets rather than of ternary precision. The gap opens along the fresh-data trajectory, saturates near 20% after three epochs, and the recipe keeps recovering the majority of it. At the E=5 best learning rate (3e-3 for the recipe): ternary 5.9256, recipe 5.8077, recovery 68%, residual +5.8%.

The penalty tracks the total budget, fresh or repeated. With epoch count and total tokens collinear by design here (the pool is fixed), we read this across experiments rather than within one: the gap grows along a trajectory statistically indistinguishable from the fresh-token curve of Section 6 at this noise floor (11.8% at 150M total vs. 8.6% at 150M fresh, a 0.029 CE difference inside the propagated noise, $\sqrt{2} \times 0.036 \approx 0.051$; 21.2% at 225M total vs. 19.4% at 300M fresh, budgets not matched), which is the super-data-learner hypothesis applied to the penalty itself: every view that teaches the model something new is also a view on which the constrained grid falls short.

Then it saturates. Between three and five epochs the gap stops growing (21.2% → 20.2%, a difference of 0.008 CE, well inside the noise floor). The plateau value matches the 341M measurement at 4B unique tokens (19.2%) despite 54× the unique data and 13× the parameters, suggesting an asymptote of roughly 20% for this architecture family and recipe-free training. We treat the universality as a hypothesis, not a conclusion: two scales and one architecture support it. The deep-epoch arms answer the durability question: the plateau holds through ten epochs (gap 21.5% at E=10, with both models still improving in parallel, −0.115 vs. −0.125 CE from E=5), and at twenty epochs the gap reads 24.5%, a +0.025 CE drift from E=10 that sits below the noise floor: a slow creep cannot be excluded, degradation can. Meanwhile the FP16 twin is *still gaining* at the twentieth pass over the same 75M tokens (5.6262 → 5.5547), beyond the point where repeated data has decayed to near-zero value for autoregressive models (Muennighoff et al., 2023).

9.3 The recovery recipe through the epochs

The recipe of Section 7 (distillation from the FP16 twin we train anyway, plus a per-channel scale at +0.07% memory) recovers 70% of the gap on fresh data. On repeated data it recovers 82% at E=2, 59% at E=3, 57% at E=5, and 39% at E=10 (residual +12.6%): recovery weakens steadily as teacher and student saturate on the same pool, and the sweet spot for the recipe is three to five epochs. With the learning rate re-tuned at E=5 (3e-3; the bare ternary optimum does not move, the recipe's does), recovery is 68% and the residual penalty is +5.8% (+8.3% at the default rate). In absolute terms the recipe keeps paying at depth, the E=10 recipe arm (5.7446) is the strongest recipe checkpoint in the ladder, but the FP16 twin it chases moves faster.

Two details deserve recording. First, at E=1 the teacher's soft targets add information beyond the one-hot labels even with no gap to recover, and the recipe lands 0.068 CE below the twin it distills from (6.3036 vs. 6.3716); the credit belongs to the extra supervision, not to ternary precision, consistent with the additive taxonomy of Section 7. Second, the learning-rate result inverts the fresh-data finding: at 300M fresh tokens the bare-ternary optimum had drifted from 4e-3 to 3e-3, while at 375M repeated tokens the bare model is indifferent and only the recipe benefits from 3e-3. Budget, not epochs, moves the optimum.

What this buys in practice. The price list, read for deployment: at a fixed memory budget, epochs buy back the quality that precision costs. The recipe at three epochs reaches 5.95 in roughly 5.4× less model memory, at the price of 3× the training tokens plus a teacher forward per step (the teacher being the FP16 twin the pipeline trains anyway). FP16 wins every budget-matched cell in Table 5, including at twenty epochs; the recipe is for the practitioner who cannot ship 682 MB.

9.4 Pre-registered technique arms: zero for three, and what that teaches

The taxonomy of Section 7 is a heuristic, not a proven law: it holds that interventions which *add* information or parameters tend to move the ternary gap while those that redistribute the existing budget tend not to, and (as this section finds) it has at least one boundary case, an epoch-aligned schedule that pays. Concretely, in the single-epoch regime the redistributive arms did not move the gap. Multi-epoch training offered a chance to use the taxonomy predictively, so before the arms ran we registered three predictions, all at E=5 against the recipe baseline (5.8315). *View scheduling* (raising the minimum mask rate on late epochs, so later passes present harder views of the same text): predicted to help. A *low-rank FP16 residual* (rank 8), adding parameters exactly where Section 5 showed the quantization residual concentrates: predicted to help. *Per-epoch LR re-warming* (restarting the cosine each epoch, SGDR-style (Loshchilov & Hutter, 2017)), which adds nothing: predicted within noise.

Every prediction was wrong. View scheduling landed exactly on the baseline (5.8292, a −0.002 delta against a 0.036 noise floor); in hindsight the strict taxonomy called it correctly and we misclassified it, since choosing among views of the same data adds nothing external. The low-rank residual moved in the right direction but below the noise floor (5.8134, −0.018), and spends 0.59 MB to do less than re-tuning the learning rate does for free (5.8077): with distillation and the per-channel scale active, the recoverable structure at this scale appears already taken. And the arm predicted to do nothing was the best recipe arm at its budget: per-epoch re-warming reached 5.7907, −0.041 against the E=5 baseline. Against the single-run noise floor (0.036) that reads as an effect; propagated for a difference of two runs the threshold is roughly 0.05, and the effect sits below it. The replication settles it: across three paired seeds, each with its own epoch-matched teacher, re-warming beats the plain-cosine recipe in every pair (−0.041, −0.025, −0.018 CE; paired mean −0.028, 2.5× the standard deviation of the paired differences). The effect is small, roughly 3% of perplexity, and free.

We report the score as it is because it sharpens the taxonomy rather than breaking it. The dichotomy was calibrated in the single-epoch regime, where training has no internal structure for a schedule to align with. Multi-epoch training has one, the epoch boundary, and the one redistribution that (borderline) pays is exactly the one aligned with it. The additive side of the taxonomy survives untouched: nothing redistributive beats the additive recipe, and the additive recipe itself appears saturated at this scale. The refined claim for the multi-epoch regime is: add information or parameters first; among redistributions, only epoch-aligned schedules are worth testing.

10 One number: the price is a near-constant offset

The results of Sections 6 and 9 compress into a single statement. Take every paired measurement in this paper past the undertrained regime (roughly eight tokens per parameter and beyond) and subtract the twin's masked cross-entropy from the ternary model's:

setting	unique tokens	offset (nats/token)
27M, fresh, 300M tokens (3-seed)	300M	0.178
27M, repeated, 3 epochs	75M	0.192
27M, repeated, 5 epochs	75M	0.184
27M, repeated, 10 epochs	75M	0.194
27M, repeated, 20 epochs	75M	0.219
341M, fresh, 4B tokens	4B	0.175
mean ± std		0.19 ± 0.016

Across 13× the parameters, 54× the unique data, and fresh versus repeated tokens, the six mature-regime pairs are described by a single relation,

$$L_{\text{tern}}(N, D) \;=\; L_{\text{fp16}}(N, D) \;+\; \delta, \qquad \delta = 0.19 \pm 0.016 \text{ nats/token}, \qquad D/N \gtrsim 8, \tag{4}$$

where L is validation masked cross-entropy, N parameters, D total training tokens, and the uncertainty is the standard deviation across the six pairs, not a confidence interval: five of the six points are 27M models and four of those share one 75M-token pool, so the pairs are not independent samples of a (scale, budget) space. Equation 4 is an empirical regularity that six paired measurements failed to break, not a fitted law; the 341M point is the only cross-scale test it has passed (each pair's own seed noise is $\hat{\sigma} = 0.013$, so the spread is barely above measurement noise). The price of ternary weights is empirically a constant δ per masked token, 0.19 ± 0.016 across the six measurements, (the twenty-epoch point sits at the high end, the within-noise creep of Section 9.2). Every headline percentage in this paper is this one number in disguise: $e^{0.19} - 1 \approx 21\%$ of perplexity. The apparent *growth* of the penalty at small budgets (8.6% at 150M tokens, 11.8% at two epochs) is the offset still ramping toward its value while the model is undertrained, which is the same regime where low-bit quantization is reported to be nearly free (Ouyang et al., 2025); the apparent *saturation* afterward is nothing but two learning curves descending in parallel at a fixed vertical distance. This contrasts with Spectra (Kaushal et al., 2025), which reports the ternary gap narrowing with scale and data at the billion-parameter, compute-optimal end; in our smaller, undertrained regime we measure a flat offset instead, by holding everything but precision fixed. The two are not in conflict if the offset is flat below the compute-optimal ratio and narrows above it, which our scales cannot test. The recovery recipe, in this reading, does one thing: it shrinks the offset, to roughly 0.06 nats at the 27M best-LR point and 0.10 post-hoc at 341M, uniformly across the denoising process (Section 8).

The reading is falsifiable, and it has already been tested once, against us. Before the from-scratch parity run completed we derived its landing point by composing Eq. 4 with the 27M recovery fraction: $0.19 \times (1 - 0.70) \approx 0.06$ nats from the teacher. The measured offset is 0.178 nats, indistinguishable from the bare model's 0.175 (Figure 7). The lesson is precise: Eq. 4 describes the *intrinsic* penalty of the ternary grid, a property of the converged fit, and says nothing about the *optimization dynamics* of recovering it, which are scale- and mode-dependent (Figure 4); composing the two, as our registered prediction did, mixes a static quantity with a dynamic one. One prediction therefore remains registered and unscored, for the run this paper leaves as future work: a 341M student distilled *post-hoc* from a multi-epoch teacher should land near the teacher plus $0.19 \times (1 - r)$ with r the post-hoc recovery measured at that scale (0.41 to 0.49), an offset of 0.10 to 0.11 nats; post-hoc is the mode whose recovery fraction we have measured at 341M, which is what rescues the composition.

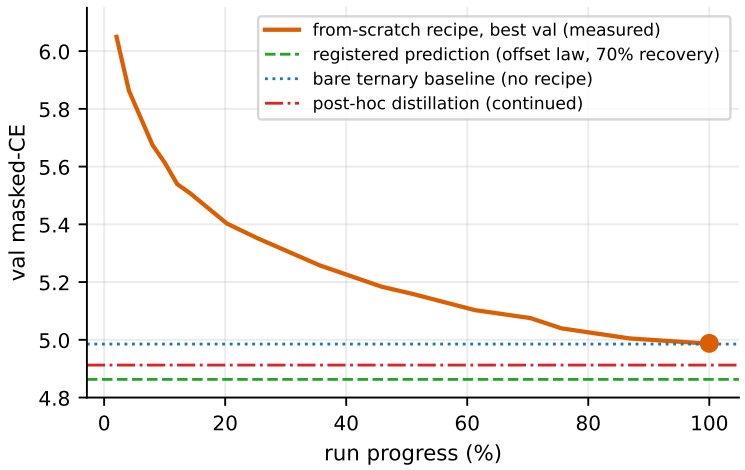

Figure 7. A registered prediction, scored against the run that tested it. The from-scratch 341M recipe run (orange curve, best validation masked-CE against run progress) was predicted by the constant-offset reading to land near 4.86 (green dashed line, teacher plus $0.19 \times (1 - 0.70)$ nats); it lands at 4.988 (marker), within noise of the bare ternary baseline (blue dotted) and above the post-hoc distillation result (red dash-dotted). The prediction fails, and the failure localizes what the offset law does and does not describe.

11 What the model can do: infilling versus composition

Perplexity is the training metric; it says nothing about what either model can actually do with its weights. We therefore evaluate the task the masked-diffusion objective trains, reconstruction of masked tokens from bidirectional context, and contrast it with free generation, which the objective never trains directly. The two results together locate what a 341M model at 12 tokens per parameter has and has not learned, and what ternary quantization costs on each.

Infilling. We mask held-out text (256 windows of 512 tokens from the validation split) at ratios from 15% to 70%, with identical seeded masks for both models, and score masked-token recovery (Table 6). Three observations. First, the models can do this: at 15% masking the FP16 twin recovers 57.7% of masked tokens exactly, on open-domain web text with a 32k vocabulary. Second, the ternary penalty on this task is 7–9% relative, far below the 19.4% suggested by perplexity: perplexity integrates the full predictive distribution, while top-1 recovery only asks whether the mode is right, and the mode survives quantization better than the tails. Third, accuracy falls steeply for both models as the mask ratio grows and context thins (57.7% to 20.7% for FP16): reconstruction degrades continuously toward composition, and the ternary gap widens slightly along the way (7.1% at ratio 0.15, 9.0% at 0.5).

	top-1 recovery			top-5 recovery	
mask ratio	ternary	FP16	rel. gap	ternary	FP16
15%	53.6%	57.7%	−7.1%	73.3%	77.3%
30%	44.8%	48.8%	−8.2%	65.7%	69.2%
50%	31.8%	35.0%	−9.0%	51.7%	54.9%
70%	19.0%	20.7%	−8.4%	36.1%	38.3%

Table 6. Masked-token recovery on held-out text at 341M, identical seeded masks for both models. The ternary model retains roughly 92% of FP16 infilling accuracy at every ratio; the perplexity gap of 19.4% overstates the cost on the task the objective actually trains.

The recovery recipe helps here more than perplexity suggests: at 341M, continued distillation closes 55 to 75% of the top-1 infilling gap across mask ratios against 41% of the CE gap, consistent with the mode of the distribution repairing before the tails. Qualitatively, errors are semantically typed rather than random:

masking *est* in "il sole sorge a est e tramonta a ovest" draws {sud, nord, ovest} from the ternary model, the right category and the wrong member; masking *pecorino* in a carbonara recipe draws {ricotta, formaggio, parmigiano} from FP16, all cheeses. Both models fail simple arithmetic (*quattro* in "due più due fa quattro"), a known weakness at this budget.

Free generation, and where it fails. At the far end of the same continuum, free generation from a short prompt, both models degenerate into repetition loops ("il Regno d'Italia è il Regno d'Italia ..." from FP16, "la capitale francese, la capitale francese ..." from the ternary model), the classic failure of likelihood-trained text generation (Holtzman et al., 2020). We quantify it as the fraction of repeated 4-grams in the generated span (rep4). The failure is a property of the budget, not of quantization: failure modes and rep4 are indistinguishable between the two models, so the 19.4% perplexity gap does not surface as a generation-quality gap; at 12 tokens per parameter neither model composes. Decoding controls help exactly as far as they can: banning repeated 3-grams and a sign-aware repetition penalty (Keskar et al., 2019) on a fully parallel confidence decoder with remasking (Chang et al., 2022; Nie et al., 2025) eliminate exact loops by construction (rep4 falls from 0.39–0.85 across prompts and decoders to 0.05–0.13) but morphological near-repeats survive, and no decoder supplies the composition the training budget did not. We also record a reranking trap: selecting the best of N samples by the model's own pseudo-likelihood actively prefers degenerate loops, because repetitive text scores high likelihood (Holtzman et al., 2020); a repetition-penalized score fixes the selection.

The practical summary for deployment is task-dependent: on the reconstruction family of tasks the model is actually good at, ternary costs 7–9% relative accuracy for a 5.4× reduction in total model memory (7.2× with the int8 embedding of Section 13). Where memory is the binding constraint the ternary model is the one to ship for infilling-style use, at that 7–9% accuracy cost; without a memory constraint, FP16 remains more accurate and is the one to ship. Free generation at this scale is not a use case for either model.

12 Stability of ternary QAT on masked diffusion

The expected failure mode is divergence, as latent weights cross rounding boundaries and sign flips propagate through attention logits. We observe none. The bin-flip rate, the fraction of ternary weights changing value per step, falls from 0.95% early to 0.14% late; the oscillation pathology that schedule- and estimator-based QAT methods target (Nagel et al., 2022) does not occur here. This is the mechanistic reason the redistributive arms of Section 7 fail: there is no instability for them to fix.

We separately test whether Muon (Jordan et al., 2024), which orthogonalizes the update matrix rather than rescaling per-coordinate, changes the picture, since the argument that custom gradient estimators collapse to plain STE relies on per-coordinate optimizer invariance that Muon lacks. Both controls ran on the matched autoregressive variant (see Limitations). Muon does not help at this scale: matched against a well-tuned AdamW baseline it is 6.5% worse on the ternary model, consistent with reports that its advantage is largest against weak baselines (Wen et al., 2025). We also find that clipping the STE gradient outside $[-s, s]$ is harmful (93% of latent weights escape the representable range and die there), confirming that plain STE, despite BitNet's "RoundClip" naming, is the correct default. A practical corollary of this stability is that early extrapolation undersells the model: Figure 8 shows the 341M ternary run finishing well below a power-law fit to its own first validation points, so killing a ternary run on an early fit abandons a model that had not finished learning.

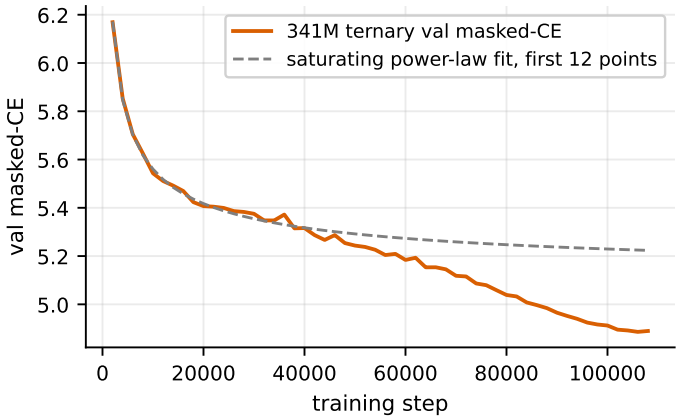

Figure 8. The 341M ternary run beats its own early power-law fit: a fit to the first 12 validation points extrapolates to 5.22, and the run finishes at 4.886. Killing a ternary run on an early fit abandons a model that had not finished learning.

13 Deployment: memory, not latency

The honest deployment claim is narrow, and getting it right is what separates this from vendor framing. Ternary weights are *smaller*, not *faster*, on a GPU at batch.

The embedding table is the memory budget. It is tempting to quote a ternary model's size as (parameters × 1.58 bits), which at 341M gives about 64 MB and is wrong by a factor of two for a different reason than rounding. The 308M block weights actually compress to 58.8 MiB packed five trits to a byte (1.6 bits each), but the 32.8M-parameter embedding cannot be ternarized and stays FP16 at 62.5 MB (Table 7). The part quantization cannot touch is larger than the part it can, so the highest-value further compression is int8 on the embedding, not any refinement of the ternary weights, and the memory advantage grows with depth relative to vocabulary (a 1B model looks far better on this accounting than a 341M one).

	params	MiB
Blocks (ternary, 1.58 b)	308.3M	58.8
Embedding (FP16)	32.8M	62.5
Total	341.1M	**121.3**
Embedding at int8	32.8M	31.3
Total (int8 emb)	341.1M	**90.1**

Table 7. On-device memory at 341M. Quantizing the embedding to int8, routine and near-lossless, saves more than any refinement to the ternary weights could. This is distinct from ternarizing the embedding, which collapses quality.

There is no ternary tensor core, and what to do about it. Current GPUs have no native ternary or sub-4-bit integer matmul; recent architectures dropped even INT4. A ternary weight-only GEMM must therefore dequantize, and at batch it is *slower* than FP16, because the win was memory bandwidth and batching moves the bottleneck to compute. The field's answer, and ours, is not to execute ternary arithmetic but to use ternary for storage and a supported integer type for compute. Microsoft's GPU kernel packs the ternary weights and feeds INT8 tensor cores (a W1.58A8 scheme), recovering the 2× that INT8 tensor cores offer over FP16 while keeping the memory saving (Mo et al., 2025). We survey these kernels rather than benchmark them ourselves; the latency claims below are the vendors' own, and only the memory arithmetic is ours. On CPU and edge, lookup-table kernels such as bitnet.cpp (Wang et al., 2024) and T-MAC (Wei et al., 2025) replace multiplication with table lookups and realize a genuine speedup in the memory-bound batch-1 decode regime, though the advantage shrinks where strong matrix hardware exists. Mixed-precision

GEMM kernels (Marlin (Frantar et al., 2025), and the ggml/GGUF TQ1_0/TQ2_0 types (Gerganov & llama.cpp contributors, 2023)) occupy the same design point. The upshot for a diffusion LM, which as Section 2.1 noted re-evaluates the full network at every denoising step with no KV cache to amortize, is that per-step compute matters more than for an autoregressive model, so the batch-1 memory-bound regime, edge deployment, and the case where the FP16 model does not fit at all are where a ternary diffusion LM is the right tool. Where it is throughput-bound on a datacenter GPU, it is smaller, not faster, and we say so.

14 Related work

Post-training quantization. GPTQ (Frantar et al., 2023) and AWQ (Lin et al., 2024) are the standard 3 to 4 bit baselines; SmoothQuant (Xiao et al., 2023) and LLM.int8() (Dettmers et al., 2022) handle activation outliers. Below 3 bits requires incoherence processing (Chee et al., 2023; Tseng et al., 2024), additive codebooks (Egiazarian et al., 2024), learned clipping (Shao et al., 2024), or keeping outlier weights in high precision (Dettmers et al., 2024; Kim et al., 2024), the last of which our residual analysis supports.

Low-bit diffusion LMs, all post-training. Quant-dLLM (Zhang et al., 2025) pushes LLaDA and Dream to 2-bit weights with a calibration-based any-order quantizer; DLLMQuant (Xu & Yang, 2025) and the study of Lin et al. (2025) cover 3 to 8 bits. None train with quantization in the loop, which is the gap this paper fills. The question is becoming a production question: diffusion LMs now ship at 3B to 14B parameters with competitive accuracy and multi-mode decoding (Fu et al., 2026), and what native low-bit training costs them is exactly what a deployment decision needs.

Ternary QAT and its frontier. BitNet b1.58 (Ma et al., 2024) established native ternary training for autoregressive LLMs and BitNet b1.58 2B4T (Ma et al., 2025) carried it to 2B parameters; TernaryLLM (Chen et al., 2024) pursues learnable ternarization; Spectra (Kaushal et al., 2025) is the reference scaling study of ternary LMs and reaches the opposite conclusion on the data axis to ours: it reports the ternary-to-full-precision gap *narrowing* toward parity at billion-parameter scale and ternary models benefiting disproportionately from more tokens. Our constant-offset result holds in a different regime (27M to 341M parameters, at or below twelve tokens per parameter); whether the offset stays flat or narrows in Spectra's compute-optimal billion-parameter regime is exactly the question our scales cannot reach. ParetoQ (Liu et al., 2025) shows that under QAT the accuracy-per-bit frontier moves from 4-bit (the PTQ optimum) toward 2-bit, which is the regime we work in. TerDiT (Lu et al., 2024) applies ternary QAT to image diffusion transformers, the closest methodological neighbour, but works on continuous latent diffusion over images rather than discrete masked denoising over tokens.

15 Limitations

Scale and seeds. The winning arms and the baseline are 3-seed with reported error bars; the redistributive arms are single-seed and judged against the measured noise floor, which is the correct instrument for a null result but a weaker one than replication. The 341M confirmation covers the baseline gap, continued distillation, and the from-scratch null result, each from a single run at that scale. The Muon and clip-STE controls were run on the autoregressive variant only and are assumed, not shown, to transfer to the diffusion objective. The 341M from-scratch failure carries two registered hypotheses, depth and maturity (Section 7.1), discriminated by a future FP16-warmup run whose design is committed to the repository. Epoch count and total budget are confounded by design in Section 9 (the pool is fixed), so "tracks the total budget" rests on the fresh-vs-repeated comparison across sections rather than a within-design control; a pool-size control at fixed budget is planned. The FP16 twin keeps its 2e-3 learning rate at every epoch count while the ternary arms get a re-tune at five epochs, so the repeated-data plateau may be slightly underestimated. At deep repetition the recipe is measured only to ten epochs. The registered predictions of Section 9.4 live as dated entries in the released results log. Two natural extensions are left as future work with their designs committed to the repository: a 341M multi-epoch run with post-hoc distillation and per-epoch re-warming, whose prediction Section 10 keeps registered and unscored, and a task-level Italian infilling benchmark. Nothing in a 27M proxy licenses a claim about 1B, which is the scale the scaling question ultimately targets.

Undertraining, and which way it biases us. At roughly 12 tokens per parameter we are far under the compute-optimal ratio (Hoffmann et al., 2022; Kaplan et al., 2020), for the practical reason that a Chinchilla-optimal 341M run is 6.8B tokens and the token-count sweep that would settle the converged gap is well beyond an independent budget. The direction of the resulting bias is the opposite of what a reader might assume, and our own data fixes it: the ternary penalty *grows* with the token budget (Section 6, 8.6% to 19.4%), consistent with (Kumar et al., 2025; Ouyang et al., 2025). So the gap we report is a *lower* bound on the converged gap, not an upper bound: training to compute-optimality would widen it, and near-parity in undertrained runs is an artifact, not a converged property. This makes the recovery result stronger rather than weaker: distillation recovers 70% of a gap that would only be larger at convergence.

One language, one tokenizer. All results are on Italian FineWeb-2 with a 32k vocabulary; the memory accounting in particular depends on the vocabulary-to-depth ratio, and the constant of Eq. 4 inherits both choices: 0.19 nats per masked token is measured under one tokenizer's granularity and one language's predictability, and a vocabulary or language with different per-token entropy may sit at a different constant. What we expect to transfer is the shape, a ramp then a near-constant offset, not the number. Ternary is also known to underperform more on web-corpus perplexity than on structured text (Kaushal et al., 2025).

Latency. Our thesis is memory, not latency. We make no test-time-compute claim: best-of-N reranking cannot improve perplexity, and reranking by the model's own likelihood is anti-correlated with quality.

16 Conclusion

A masked diffusion language model can be trained from scratch with ternary weights, and the price is now measured rather than guessed. Post-training quantization is not the alternative at this precision: it collapses even when calibrated. The native penalty is one number with a domain: below roughly eight tokens per parameter it is still ramping, and past that it is empirically a near-constant 0.19 nats per masked token across our six paired measurements, about 21% of perplexity, at every scale, budget, and freshness we tested; it lives where prediction depends on rich context. Two additions, distillation from the full-precision twin the pipeline trains anyway and a per-channel scale, buy back two thirds of it at the proxy scales and up to half at 341M, with a mode distinction we believe is previously unreported: post-hoc distillation scales and from-scratch distillation does not, and everything that merely rearranges training buys nothing, a dichotomy that survived every arm we ran and failed every attempt we pre-registered to extend it. On a finite corpus the practical recipe is short: train the twin and the bare ternary model, repeat the corpus three to five times, distill the finished ternary from the twin, and ship it in 5.4× less memory, if memory is the constraint; train FP16 if it is not.

A Experimental details

Data and tokenizer. Italian FineWeb-2, tokenized with a 32k-vocabulary SentencePiece unigram model trained on the same corpus (vocabulary 32,001 with the mask token at id 32,000). Validation is the final 1% of the token stream; the held-out test split of Section 7.1 is the preceding 1%, used by no run and no selection decision; the out-of-domain set is 5M tokens of Italian Wikipedia tokenized with the same model.

27M proxy runs. d_{model}=448, 8 layers, 7 heads, d_{ff}=1216, sequence length 512, batch 32 (16,384 tokens per step). AdamW, β_1=0.9, β_2=0.95, weight decay 0.1 annealed to 0 at two thirds of training, no weight decay on the ternary latent weights. Warmup 200 steps, then a single cosine to 10% of peak. Peak learning rate 4e-3 (ternary) and 2e-3 (FP16) unless an arm states otherwise. Mask rate $t \sim U(10^{-3}, 1]$. Distillation arms: KD weight 0.5, temperature 2.0, epoch-matched FP16 teacher, teacher not compiled. Validation: 40 fixed batches with seeded windows and masks (seed 1234), identical across arms. Hardware: one RTX 3090.

341M runs. d_{model}=1024, 24 layers, 16 heads, d_{ff}=2816, tied embeddings, vocabulary as above. The baseline twins train on 4B tokens in 108,505 steps; continued distillation adds 1B tokens at LR 1e-3, batch 8, sequence length 1024; the from-scratch recipe run uses LR 4e-3 at the same batch geometry for the full

4B tokens. Same optimizer family and schedule as the proxy. Hardware: one L40S (48 GB); `torch.compile` on the student only (compiling the frozen teacher deadlocked a run, an operational note we pass along). Common-metric re-evaluations use one protocol for every checkpoint: 40 batches, batch 8, sequence length 1024, seed 1234.

Code and data availability. Training, evaluation, and analysis code, the per-run result tables backing every number, and the model checkpoints (four 341M models including the FP16 twin and the from-scratch null control, plus the 27M research ladder) are released. Code and result tables: `https://github.com/lupodevelop/echo-1.58`. Model checkpoints: the Hugging Face collection `https://huggingface.co/collections/lupodevelop/echo-158-ternary-masked-diffusion-lms-6a5dd179a5f7ad08cc277404`. A versioned snapshot is archived at Zenodo, DOI: 10.5281/zenodo.21455055.

A throughput trap. Ternary QAT is not compute-bound. Eq. 3 expands into a chain of unfused elementwise operations on every weight and activation of every layer on every forward pass. Uncompiled, this makes the model memory-bandwidth-bound and throughput *falls* with batch size (14.0k tokens/s at batch 4, 10.7k at batch 12). Under `torch.compile`, which fuses the chain, the same model reaches 41.1k tokens/s at batch 12 and peak memory drops from 38.2 to 25.0 GiB. The consequence is a factor of four in cost, and the trap is that an uncompiled probe reports an ETA four times too pessimistic; none of it is visible in the model definition.

References

Jacob Austin, Daniel D. Johnson, Jonathan Ho, Daniel Tarlow, and Rianne van den Berg. Structured denoising diffusion models in discrete state-spaces. In Marc'Aurelio Ranzato, Alina Beygelzimer, Yann N. Dauphin, Percy Liang, and Jennifer Wortman Vaughan (eds.), *Advances in Neural Information Processing Systems 34 (NeurIPS 2021)*, pp. 17981–17993, 2021.

Yoshua Bengio, Nicholas Léonard, and Aaron C. Courville. Estimating or propagating gradients through stochastic neurons for conditional computation, 2013.

Huiwen Chang, Han Zhang, Lu Jiang, Ce Liu, and William T. Freeman. MaskGIT: Masked generative image transformer. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022. arXiv:2202.04200.

Jerry Chee, Yaohui Cai, Volodymyr Kuleshov, and Christopher De Sa. QuIP: 2-bit quantization of large language models with guarantees. In Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine (eds.), *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023.

Tianqi Chen, Zhe Li, Weixiang Xu, Zeyu Zhu, Dong Li, Lu Tian, Emad Barsoum, Peisong Wang, and Jian Cheng. TernaryLLM: Ternarized large language model, 2024.

Jang Hyun Cho and Bharath Hariharan. On the efficacy of knowledge distillation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 4794–4802, 2019.

Jungwook Choi, Zhuo Wang, Swagath Venkataramani, Pierce I-Jen Chuang, Vijayalakshmi Srinivasan, and Kailash Gopalakrishnan. PACT: Parameterized clipping activation for quantized neural networks, 2018.

Alexander Conzelmann, Albert Catalan-Tatjer, and Shiwei Liu. Layer collapse in diffusion language models, 2026.

Matthieu Courbariaux, Yoshua Bengio, and Jean-Pierre David. BinaryConnect: Training deep neural networks with binary weights during propagations. In Corinna Cortes, Neil D. Lawrence, Daniel D. Lee, Masashi Sugiyama, and Roman Garnett (eds.), *Advances in Neural Information Processing Systems 28 (NIPS 2015)*, pp. 3123–3131, 2015.

Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for transformers at scale. In Sanmi Koyejo, S. Mohamed, A. Agarwal, Danielle Belgrave, K. Cho, and A. Oh (eds.), *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, 2022.

Tim Dettmers, Ruslan Svirschevski, Vage Egiazarian, Denis Kuznedelev, Elias Frantar, Saleh Ashkboos, Alexander Borzunov, Torsten Hoefler, and Dan Alistarh. SpQR: A sparse-quantized representation for near-lossless LLM weight compression. In *The Twelfth International Conference on Learning Representations (ICLR)*. OpenReview.net, 2024.

Dayou Du, Yijia Zhang, Shijie Cao, Jiaqi Guo, Ting Cao, Xiaowen Chu, and Ningyi Xu. BitDistiller: Unleashing the potential of sub-4-bit LLMs via self-distillation. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar (eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 102–116. Association for Computational Linguistics, 2024. DOI: 10.18653/v1/2024.acl-long.7.

Vage Egiazarian, Andrei Panferov, Denis Kuznedelev, Elias Frantar, Artem Babenko, and Dan Alistarh. Extreme compression of large language models via additive quantization. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*, pp. 12284–12303. PMLR, 2024.

Steven K. Esser, Jeffrey L. McKinstry, Deepika Bablani, Rathinakumar Appuswamy, and Dharmendra S. Modha. Learned step size quantization. In *8th International Conference on Learning Representations (ICLR)*. OpenReview.net, 2020.

Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *The Eleventh International Conference on Learning Representations (ICLR)*. OpenReview.net, 2023.

Elias Frantar, Roberto L. Castro, Jiale Chen, Torsten Hoefler, and Dan Alistarh. MARLIN: Mixed-precision auto-regressive parallel inference on large language models. In *Proceedings of the 30th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*, pp. 239–251, 2025. DOI: 10.1145/3710848.3710871.

Yonggan Fu, Lexington Whalen, Abhinav Garg, Chengyue Wu, Maksim Khadkevich, Nicolai Oswald, Enze Xie, Daniel Egert, et al. Nemotron-labs-diffusion: A tri-mode language model unifying autoregressive, diffusion, and self-speculation decoding, 2026.

Georgi Gerganov and llama.cpp contributors. llama.cpp: LLM inference in C/C++, 2023. ggml/GGUF ecosystem with TQ1_0/TQ2_0 ternary types.

Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katherine Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Oriol Vinyals, Jack W. Rae, and Laurent Sifre. An empirical analysis of compute-optimal large language model training. In Sanmi Koyejo, S. Mohamed, A. Agarwal, Danielle Belgrave, K. Cho, and A. Oh (eds.), *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, 2022.

Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. In *The Eighth International Conference on Learning Representations (ICLR)*, 2020. arXiv:1904.09751.

Keller Jordan, Yuchen Jin, Vlado Boza, Jiacheng You, Franz Cesista, Laker Newhouse, and Jeremy Bernstein. Muon: An optimizer for hidden layers in neural networks, 2024. Blog post, `https://kellerjordan.github.io/posts/muon/`.

Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models, 2020.

Ayush Kaushal, Tejas Vaidhya, Arnab Kumar Mondal, Tejas Pandey, Aaryan Bhagat, and Irina Rish. Surprising effectiveness of pretraining ternary language models at scale. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025. arXiv:2407.12327, arXiv title prefixed Spectra:.

Nitish Shirish Keskar, Bryan McCann, Lav R. Varshney, Caiming Xiong, and Richard Socher. CTRL: A conditional transformer language model for controllable generation, 2019.

Sehoon Kim, Coleman Hooper, Amir Gholami, Zhen Dong, Xiuyu Li, Sheng Shen, Michael W. Mahoney, and Kurt Keutzer. SqueezeLLM: Dense-and-sparse quantization. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*, pp. 23901–23923. PMLR, 2024.

Taku Kudo and John Richardson. SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In Eduardo Blanco and Wei Lu (eds.), *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP)*, pp. 66–71. Association for Computational Linguistics, 2018. DOI: 10.18653/v1/d18-2012.

Tanishq Kumar, Zachary Ankner, Benjamin F. Spector, Blake Bordelon, Niklas Muennighoff, Mansheej Paul, Cengiz Pehlevan, Christopher Ré, and Aditi Raghunathan. Scaling laws for precision. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025. arXiv:2411.04330.

Haokun Lin, Haobo Xu, Yichen Wu, Ziyu Guo, Renrui Zhang, Zhichao Lu, Ying Wei, Qingfu Zhang, and Zhenan Sun. Quantization meets dLLMs: A systematic study of post-training quantization for diffusion LLMs, 2025.

Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. In Phillip B. Gibbons, Gennady Pekhimenko, and Christopher De Sa (eds.), *Proceedings of the Seventh Annual Conference on Machine Learning and Systems (MLSys)*. mlsys.org, 2024.

Bin Liu, Fengfu Li, Xiaoxing Wang, Bo Zhang, and Junchi Yan. Ternary weight networks. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 1–5. IEEE, 2023. DOI: 10.1109/ICASSP49357.2023.10094626.

Zechun Liu, Changsheng Zhao, Hanxian Huang, Sijia Chen, Jing Zhang, Jiawei Zhao, Scott Roy, Lisa Jin, Yunyang Xiong, Yangyang Shi, Lin Xiao, Yuandong Tian, Bilge Soran, Raghuraman Krishnamoorthi, Tijmen Blankevoort, and Vikas Chandra. ParetoQ: Improving scaling laws in extremely low-bit LLM quantization. In *Advances in Neural Information Processing Systems 38 (NeurIPS)*, 2025. arXiv:2502.02631.

Ilya Loshchilov and Frank Hutter. SGDR: Stochastic gradient descent with warm restarts. In *The Fifth International Conference on Learning Representations (ICLR)*, 2017. arXiv:1608.03983.

Xudong Lu, Aojun Zhou, Ziyi Lin, Qi Liu, Yuhui Xu, Renrui Zhang, Yafei Wen, Shuai Ren, Peng Gao, Junchi Yan, and Hongsheng Li. TerDiT: Ternary diffusion models with transformers, 2024.

Shuming Ma, Hongyu Wang, Lingxiao Ma, Lei Wang, Wenhui Wang, Shaohan Huang, Li Dong, Ruiping Wang, Jilong Xue, and Furu Wei. The era of 1-bit LLMs: All large language models are in 1.58 bits, 2024.

Shuming Ma, Hongyu Wang, Shaohan Huang, Xingxing Zhang, Ying Hu, Ting Song, Yan Xia, and Furu Wei. BitNet b1.58 2b4t technical report, 2025.

Seyed Iman Mirzadeh, Mehrdad Farajtabar, Ang Li, Nir Levine, Akihiro Matsukawa, and Hassan Ghasemzadeh. Improved knowledge distillation via teacher assistant. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pp. 5191–5198, 2020.

Zhiwen Mo, Lei Wang, Jianyu Wei, Zhichen Zeng, Shijie Cao, Lingxiao Ma, Naifeng Jing, Ting Cao, Jilong Xue, Fan Yang, and Mao Yang. LUT tensor core: A software-hardware co-design for LUT-based low-bit LLM inference. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA)*, 2025. DOI: 10.1145/3695053.3731057.

Niklas Muennighoff, Alexander M. Rush, Boaz Barak, Teven Le Scao, Aleksandra Piktus, Nouamane Tazi, Sampo Pyysalo, Thomas Wolf, and Colin Raffel. Scaling data-constrained language models. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023. arXiv:2305.16264.

Markus Nagel, Marios Fournarakis, Yelysei Bondarenko, and Tijmen Blankevoort. Overcoming oscillations in quantization-aware training. In *Proceedings of the 39th International Conference on Machine Learning (ICML)*, volume 162 of *PMLR*, 2022. arXiv:2203.11086.

Jinjie Ni, Qian Liu, Longxu Wang, Chao Du, Hang Ye, Zili Xu, Tianyu Pang, Fuzhao Du, et al. Diffusion language models are super data learners, 2025.

Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large language diffusion models. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*, 2025.

Xu Ouyang, Tao Ge, Thomas Hartvigsen, Zhisong Zhang, Haitao Mi, and Dong Yu. Low-bit quantization favors undertrained LLMs. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025. arXiv:2411.17691.

Guilherme Penedo, Hynek Kydlíček, Vinko Sabolčec, Bettina Messmer, Negar Foroutan, Amir Hossein Kargaran, Colin Raffel, Martin Jaggi, Leandro von Werra, and Thomas Wolf. FineWeb2: One pipeline to scale them all — adapting pre-training data processing to every language. In *Proceedings of the Second Conference on Language Modeling (COLM)*, 2025.

Mihir Prabhudesai, Mengning Wu, Amir Zadeh, Katerina Fragkiadaki, and Deepak Pathak. Diffusion beats autoregressive in data-constrained settings, 2025.

Mohammad Rastegari, Vicente Ordonez, Joseph Redmon, and Ali Farhadi. XNOR-Net: ImageNet classification using binary convolutional neural networks. In Bastian Leibe, Jiri Matas, Nicu Sebe, and Max Welling (eds.), *Computer Vision — ECCV 2016, 14th European Conference, Amsterdam, The Netherlands, Proceedings, Part IV*, volume 9908 of *Lecture Notes in Computer Science*, pp. 525–542. Springer, 2016. DOI: 10.1007/978-3-319-46493-$0_3$2.

Subham Sekhar Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin T. Chiu, Alexander M. Rush, and Volodymyr Kuleshov. Simple and effective masked diffusion language models. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, 2024.

Wenqi Shao, Mengzhao Chen, Zhaoyang Zhang, Peng Xu, Lirui Zhao, Zhiqian Li, Kaipeng Zhang, Peng Gao, Yu Qiao, and Ping Luo. OmniQuant: Omnidirectionally calibrated quantization for large language models. In *The Twelfth International Conference on Learning Representations (ICLR)*. OpenReview.net, 2024.

Jiaxin Shi, Kehang Han, Zhe Wang, Arnaud Doucet, and Michalis K. Titsias. Simplified and generalized masked diffusion for discrete data. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, 2024.

Albert Tseng, Jerry Chee, Qingyao Sun, Volodymyr Kuleshov, and Christopher De Sa. QuIP#: Even better LLM quantization with hadamard incoherence and lattice codebooks. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*, pp. 48630–48656. PMLR, 2024.

Jinheng Wang, Hansong Zhou, Ting Song, Shaoguang Mao, Shuming Ma, Hongyu Wang, Yan Xia, and Furu Wei. 1-bit AI infra: Part 1.1, fast and lossless BitNet b1.58 inference on CPUs, 2024. Microsoft `bitnet.cpp`.

Jianyu Wei, Shijie Cao, Ting Cao, Lingxiao Ma, Lei Wang, Yanyong Zhang, and Mao Yang. T-MAC: CPU renaissance via table lookup for low-bit LLM deployment on edge. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys)*, 2025. DOI: 10.1145/3689031.3696099.

Kaiyue Wen, David Hall, Tengyu Ma, and Percy Liang. Fantastic pretraining optimizers and where to find them, 2025.

Guangxuan Xiao, Ji Lin, Mickaël Seznec, Hao Wu, Julien Demouth, and Song Han. SmoothQuant: Accurate and efficient post-training quantization for large language models. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *Proceedings of Machine Learning Research*, pp. 38087–38099. PMLR, 2023.

Chen Xu and Dawei Yang. DLLMQuant: Quantizing diffusion-based large language models, 2025.

Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng Kong. Dream 7b: Diffusion large language models, 2025.

Tianao Zhang, Zhiteng Li, Xianglong Yan, Haotong Qin, Yong Guo, and Yulun Zhang. Quant-dLLM: Post-training extreme low-bit quantization for diffusion large language models, 2025.